STUDY PROJECT — PHASE 3

Operational Complexity Meter

Quantifying Operational Complexity in Distributed Systems

Aabid Ali Sofi | 2023EBCS041
Advisor: Preethy P Johny | Feb 2026

Agenda

1. Problem Statement
2. What is OCM?
3. The 6 Metrics
4. System Architecture
5. Pipeline Deep Dive

1. Parser & Extraction
2. Graph Algorithms
3. Storage & API
4. Dashboard & Visualization
5. Demo, Results & Future Work

SECTION

Problem Statement

The Problem

Modern distributed systems are operationally complex, but that complexity is rarely measured.

Symptoms

- Hundreds of config parameters
- Deep dependency chains
- Frequent config changes
- Env-specific drift (dev vs prod)
- Exposed failure surfaces

Impact

- Outages from config errors
- Slow incident response
- Hidden coupling between services
- Unmeasured toil & risk

Can we quantify operational complexity as a single, measurable score?

SECTION

What is OCM?

Operational Complexity Meter

OCM is a single Go binary that:

1. Scans a Kubernetes YAML repository
2. Identifies services from directory structure or manifest metadata
3. Computes 6 operational complexity metrics per service
4. Normalizes and produces a composite OCM score (0–1)
5. Stores results in SQLite for time-series tracking

6 Serves an interactive web dashboard

```
go run ./cmd/ocm --repo /path/to/k8s-repo --db ocm.sqlite --port 8080
```

No external dependencies. No K8s SDK. No CGO. Pure Go.

SECTION

The 6 Metrics

Metric Overview

Metric	What It Measures	How
CSA	Configuration Surface Area	Count of env vars, ports, resources, replicas, spec keys
DD	Dependency Depth	Longest path in the SCC-condensed dependency DAG
DB	Dependency Breadth	Direct upstream + downstream dependency count
CV	Change Volatility	Git commits affecting config in the last 30 days

FE	Failure Exposure	Exposed endpoints (LB, NodePort, Ingress, hostPort, ext URLs)
CDR	Config Drift Risk	Env-specific overrides (dev/staging/prod directory diffs)

Composite OCM Score

Normalization — Min-max across the service cohort:

$$\text{Normalized}(M) = \frac{M - M_{\text{min}}}{M_{\text{max}} - M_{\text{min}}}$$

Composite Score — Weighted sum of normalized metrics:

$$\mathrm{OCM} = \frac{1}{6} \cdot \mathrm{CSA} + \frac{1}{6} \cdot \mathrm{DD} + \frac{1}{6} \cdot \mathrm{DB} + \frac{1}{6} \cdot \mathrm{CV} + \frac{1}{6} \cdot \mathrm{FE} + \frac{1}{6} \cdot \mathrm{CDR}$$

0.0

Minimal complexity

Weights are configurable
(default: equal 1/6 each)

1.0

Maximum in cohort

SECTION

System Architecture

Single-Binary Architecture

```
┌──────────────────────────────────────────────────────────┐
│                      ocm (single binary)                 │
│                                                          │
│   ┌──────────┐    ┌──────────┐    ┌──────────────────┐  │
│   │ CLI/Main │──▶ │ Pipeline │──▶ │  SQLite Storage  │  │
│   └──────────┘    │          │    └──────────────────┘  │
│                   │ Parser   │            │             │
│                   │ GitLog   │            ▼             │
│                   │ Metrics  │    ┌──────────────────┐  │
│                   │ Scoring  │    │  HTTP API Server │  │
│                   └──────────┘    │  + Embedded      │  │
│                                   │    Dashboard     │  │
│                                   └──────────────────┘  │
└──────────────────────────────────────────────────────────┘
```

2 Dependencies

`yaml.v3`
`modernc.org/sqlite`

No CGO

Pure Go SQLite
Cross-compilable

Embedded Assets

`go:embed` HTML/CSS/JS
Zero external files

Data Flow Pipeline

1. Discovery — Walk filesystem, collect `.yaml`/`.yml`, skip `.git`/`vendor`
2. Parsing — Multi-doc YAML decode, extract CSA facts + deps + FE signals
3. Service ID — Map files to services via `dir` or `manifest` (metadata.name)
4. Metric Engine — Compute DD (graph), DB (degree), CV (git), FE, CDR
5. Normalization — Min-max across cohort for each metric
6. Composite — Weighted sum to single OCM score per service

7. Persist — Atomic transaction: services + metrics + evidence to SQLite
8. Serve — HTTP API + embedded dashboard at `localhost:8080`

SECTION

Parser & Extraction

CSA Extraction

For each Kubernetes manifest, count configuration surface contributors:

Component	Source	Count
`env`	Container environment variables	1 per var
`port`	Container port entries	1 per port
`resource`	Resource limits/requests leaf keys	1 per key
`replica`	`spec.replicas` field	1 if present
`spec_key`	Top-level keys under `spec`	1 per key

Each contributor is stored as an evidence item with source path, manifest kind, and manifest name for full traceability.

Dependency Extraction

Three heuristic detection rules for inter-service dependencies:

Rule 1 — Env var suffix matching

`_SERVICE`, `_SERVICE_HOST`, `_SERVICE_PORT`, `_ADDR`, `_HOST`, `_URL`, `_ENDPOINT`, `_URI`

Rule 2 — Cluster FQDN detection

Values containing `.svc.cluster.local` — extract service name

Rule 3 — Hostname:port pattern

Regex: `^[a-z][a-z0-9-]{0,62}:\d{2,5}$` — extract hostname

Designed for real-world repos: Sock Shop, Google Online Boutique, Helm charts, Rails/Django/Spring deployments.

Failure Exposure Detection

FE counts exposed endpoints and external integrations — 5 detection rules:

1. LoadBalancer / NodePort service types — each port = 1 endpoint
2. Ingress resources — each `rules[].http.paths[]` = 1 endpoint
3. ExternalName services — 1 external integration
4. Container hostPort — each hostPort = 1 endpoint
5. External URLs in env vars — `http(s)://` not containing `.svc.cluster.local`

Detection rules are explicitly documented in code per spec requirement.

SECTION

Graph Algorithms

Dependency Depth — Tarjan's SCC

DD = longest path (edge count) in the dependency graph.

Problem

Dependency graphs may have cycles (A → B → A).

Solution — SCC Condensation

1. Tarjan's algorithm finds SCCs

Example

```
A → B → C     (B ↔ A = cycle)
    ↑ ↙
    A

SCC: {A, B} → node S₁
DAG: S₁ → C
DD[A] = DD[B] = 1
DD[C] = 0
```

2. Condense each SCC to a single node
3. Result is a DAG (no cycles)
4. DFS with memoization → longest path

Deterministic — same input always produces same DD values.

SECTION

Storage & API

SQLite Schema

4 tables, atomic transactions, time-series support:

Core Tables

```
services(id, name, repository)
  UNIQUE(name, repository)

metrics(id, service_id,
  metric_type, metric_value,
  timestamp)

composite_scores(id, service_id,
  ocm_score, timestamp)
```

Evidence Table

```
metric_evidence(id, service_id,
  metric_type, component,
  evidence_key, evidence_value,
  source_path,
  manifest_kind, manifest_name,
  timestamp)
```

- FK constraints with `PRAGMA foreign_keys`
- Indexes on `(service_id, metric_type, timestamp)`
- Upsert for idempotent re-runs

REST API

Endpoint	Returns
`GET /api/healthz`	Health check
`GET /api/services`	List all analyzed services
`GET /api/overview`	Repo aggregates + per-service summaries
`GET /api/services/{id}/scores`	OCM score time series
`GET /api/services/{id}/metrics/{type}`	Metric time series
`GET /api/services/{id}/metrics/{type}/evidence`	Evidence items for latest run

Go 1.22 `http.ServeMux`
method + path patterns

JSON responses
proper `Content-Type`

CORS enabled
for local development

SECTION

Dashboard

Dashboard — Repository Overview

OCM Operational Complexity Meter

NAVIGATION

- Overview
- Services

SERVICES

- All Services
- api
- cache
- cart
- catalog
- db
- gateway
- payment
- web
- worker

Phase 3

Repository Overview

All Services (Repo) | Refresh

COMPOSITE OCM SCORE

0.306

Average across 9 services

SERVICES ANALYZED

9

API connected

Metric Breakdown

Aggregated across all services. Click a tile for details.

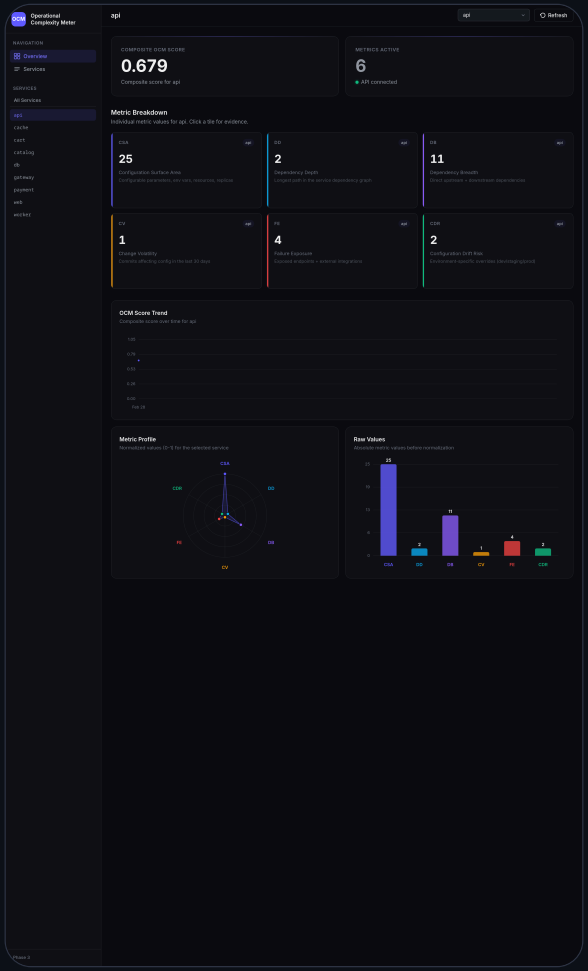
CSA	9 svc	**18.11**	Configuration Surface Area avg across repo · total: 163
DD	9 svc	**1.78**	Dependency Depth avg across repo · total: 16
DB	9 svc	**5.89**	Dependency Breadth avg across repo · total: 53
CV	9 svc	**1**	Change Volatility avg across repo · total: 9
FE	9 svc	**2.22**	Failure Exposure avg across repo · total: 20
CDR	9 svc	**0.67**	Configuration Drift Risk avg across repo · total: 6

OCM Score Trend

Select a service to see score trends

Repository overview showing composite OCM score, 9 analyzed services, and the 6 metric breakdown tiles

Dashboard — Service Detail


OCM
Operational Complexity Meter
NAVIGATION
Overview
Services
SERVICES
All Services
api
cache
cert
catalog
db
gateway
payment
web
worker
Phase 2
api
api
Refresh
COMPOSITE OCM SCORE
0.679
Composite score for api
METRICS ACTIVE
6
API connected
Metric Breakdown
Individual metric values for api. Click a tile for evidence.
CSA
api
25
Configuration Surface Area
Configurable parameters, env vars, resources, replicas
DD
api
2
Dependency Depth
Longest path in the service dependency graph
DB
api
11
Dependency Breadth
Direct upstream + downstream dependencies
CV
api
1
Change Volatility
Commits affecting config in the last 30 days
FE
api
4
Failure Exposure
Exposed endpoints + external integrations
CDR
api
2
Configuration Drift Risk
Environment-specific overrides (dev/staging/prod)
OCM Score Trend
Composite score over time for api
1.05
0.79
0.53
0.26
0.00
Feb 28
Metric Profile
Normalized values (0–1) for the selected service
CSA
DD
DB
CV
FE
CDR
Raw Values
Absolute metric values before normalization
25
20
15
10
5
0
25
2
11
1
4
2
CSA
DD
DB
CV
FE
CDR

Single service view: individual metrics, OCM score trend, radar chart, and bar chart for the `api` service

Dashboard — Service Comparison

OCM Operational Complexity Meter

NAVIGATION

- Overview
- Services

SERVICES

- All Services
- api
- cache
- cart
- catalog
- db
- gateway
- payment
- web
- worker

Phase 3

Services

api | Refresh

Service Comparison

All services with their latest metric values and composite OCM score

SERVICE	CSA	DD	DB	CV	FE	CDR	OCM
api	25	2	11	1	4	2	0.679
cache	12	0	5	1	0	0	0.024
cart	16	1	4	1	0	0	0.093
catalog	21	1	8	1	2	0	0.300
db	17	0	5	1	1	1	0.195
gateway	23	4	4	1	7	2	0.641
payment	16	2	5	1	1	0	0.182
web	18	3	6	1	2	1	0.380
worker	15	3	5	1	3	0	0.259

All 9 services compared side-by-side with raw metric values and composite OCM scores

Dashboard — Evidence Drill-Down

CSA — Configuration Surface Area

api

25 evidence items from the latest analysis run

COMPONENT	KEY	VALUE	MANIFEST	SOURCE
spec_key	replicas		Deployment / api	/home/aa
spec_key	selector		Deployment / api	/home/aa
spec_key	template		Deployment / api	/home/aa
replica	spec.replicas	3	Deployment / api	/home/aa
env	PORT	8080	Deployment / api	/home/aa
env	DB_HOST	db	Deployment / api	/home/aa
env	DB_PORT	5432	Deployment / api	/home/aa
env	CACHE_ADDR	cache:6379	Deployment / api	/home/aa
env	CART_SERVICE_ADDR	cart:4000	Deployment / api	/home/aa
env	PAYMENT_SERVICE_ENDPOINT	http://payment:8080/v1	Deployment / api	/home/aa
env	CATALOG_SERVICE_URL	http://catalog:5000	Deployment / api	/home/aa
env	NOTIFICATION_SERVICE_URI	http://notification.messaging.svc.cluster.local:8080	Deployment / api	/home/aa

CSA evidence modal for the `api` service — 25 configuration contributors with source traceability

Dashboard Design

Views & Visualizations

- Composite OCM score hero card
- 6 metric tiles with color coding
- OCM score trend chart (Canvas)
- Radar chart (normalized profile)
- Bar chart (raw metric values)
- Service comparison table
- Evidence drill-down modal

Technical Implementation

- Stripe-inspired dark design system
- Pure HTML/CSS/JS (no frameworks)
- Canvas-based chart rendering
- Responsive layout (mobile to desktop)
- Embedded via `go:embed`
- Sidebar with live service selector
- All Services vs individual service toggle

Evidence-Based Traceability

Every metric contribution is tracked with provenance:

Metric	Evidence Type	Example
CSA	`env`, `port`, `resource`, `replica`	`PORT=8080` from `api/deploy.yaml`
DD	`dep_chain`, `direct_dep`	`api -> cache -> db` (depth=2)
DB	`upstream_dep`, `downstream_dep`	`orders -> payment` edge
CV	`commit`	`commit abc12345`
FE	`loadbalancer`, `ingress`, `external_url`	LoadBalancer port 80

CDR	`env_override`	`config.yaml` in 2 envs: dev, prod

Click any metric tile in the dashboard to see exactly what contributed and where it was found.

SECTION

Testing & Validation

Test Suite

Pipeline Tests

- DD cycle handling (Tarjan's SCC)
- DB in/out degree counting
- Self-dependency ignored
- Normalization edge cases (max = min)
- All 6 metrics present in output
- Composite score in [0, 1]
- DD evidence generation

Parser Tests

- 7 dependency extraction tests
- 6 FE detection tests
- Suffix, FQDN, hostname:port patterns
- Multi-document YAML

Storage & Git Tests

- Round-trip for all 6 metric types
- 6 git log parsing tests

All 29 tests pass – Build clean – Vet clean

Project Structure

```
ocm/
├── cmd/ocm/main.go        # CLI entry
├── internal/
│   ├── model/model.go     # Types
│   ├── parser/
│   │   ├── yamlscan.go    # K8s parser
│   │   └── parser_test.go
│   ├── pipeline/
│   │   ├── pipeline.go    # Orchestration
│   │   └── pipeline_test.go
│   ├── gitlog/
│   │   ├── gitlog.go      # CV metric
│   │   └── gitlog_test.go
```

```
│   ├── storage/
│   │   ├── storage.go     # SQLite layer
│   │   └── storage_test.go
│   ├── api/api.go         # HTTP API
│   └── dashboard/
│       ├── dashboard.go   # go:embed
│       └── static/
│           ├── index.html
│           ├── styles.css
│           └── app.js
├── specs/                  # 8 spec docs
├── docs/                   # Phase docs
└── go.mod                  # 2 deps
```

SECTION

Project Evolution

Phase 2 → Phase 3

Phase 2 — MVP/PoC

- 2 metrics: CSA + DD only
- Basic dependency extraction
- Simple dashboard
- CSA evidence only
- Composite: 50/50 weights
- Manual validation

Phase 3 — Current

- All 6 metrics: CSA, DD, DB, CV, FE, CDR
- Broadened dependency detection (9 patterns)
- Git integration for CV
- FE + CDR detection rules
- Evidence for all 6 metrics
- Stripe-inspired dashboard redesign
- Radar, bar, trend, sparkline charts
- 29 automated tests
- `--cv-window` CLI flag

SECTION

Demo & Results

Running OCM

Analyze a Kubernetes repository

```
# Build and run
go run ./cmd/ocm \
  --repo /path/to/microservices-demo \
  --db ocm.sqlite --port 8080

# Output
ocm: analyzed 12 services, db=ocm.sqlite
ocm: listening on http://127.0.0.1:8080
```

JSON output mode

Configuration flags

```
go run ./cmd/ocm --repo ./repo \
  --print 2>/dev/null | jq '.'
```

```
--service-key manifest  # metadata.name
--cv-window 60          # 60-day lookback
```

Known Limitations

By Design (Prototype Scope)

- Heuristic-based extraction
- No Helm template rendering
- No Dockerfile parsing
- Single-run-on-startup model
- `git` binary required for CV
- Localhost only (no auth)

Mitigations

- Evidence trail enables manual verification
- Multiple service ID strategies
- Best-effort approach (warnings, not failures)
- Modular architecture enables extension
- Configurable weights for composite scoring

SECTION

Future Work

Potential Extensions

Near-Term

- Helm v3 template rendering
- Dockerfile complexity parsing
- Scheduled/on-demand re-analysis
- Configurable weights via UI
- Export reports (PDF/CSV)

Long-Term

- CI/CD integration (PR complexity diffs)
- Multi-repo support
- Complexity budget enforcement
- Runtime metrics correlation
- Team/org-level dashboards
- Authentication & multi-user

Summary

- 6 metrics quantifying operational complexity from K8s YAML
- Single binary — Go + embedded SQLite + embedded dashboard
- Evidence-based — every metric contribution is traceable
- Graph algorithms — Tarjan's SCC for cycle-safe dependency depth
- Minimal dependencies — only `yaml.v3` + pure-Go SQLite
- Interactive dashboard — repo overview, service drill-down, charts

+ 29 automated tests — parser, pipeline, storage, git log

Thank You

Questions?

Aabid Ali Sofi | 2023EBCS041
Operational Complexity Meter — Phase 3